Running things

Running Things

Booting Up

How to setup 481-Frontend with a connection to the backend.

Requirements:

- WVU SSH approval to your <username>@ssh.wvu.edu account.
  - https://wvu.atlassian.net/servicedesk/customer/portal/5/create/638?src=182487431
- Access to the PICARD github organization.
- Credentials to access the PICARD server from the WVU's SSH center.
- NPM on your local device.

1. Clone "481-front-end" to your local machine.
2. Install requisite libraries (npm, npm vite).
3. Start front-end with "npm run dev" after navigating to the "picard" folder. Usually under "/481-FrontEnd/picard". To access the front-end webpage press "o + enter"
   At this point the webpage WILL NOT work until you tunnel into the backend. If you want to switch browsers, just copy the URL (localhost:5173) into the other one.

3.1. Run "npm install vite" if necessary

4. Open another terminal and SSH to WVU's ssh center using either:

```
ssh -L 5080:localhost:5080 <username>@ssh.wvu.edu
ssh -L 5080:127.0.0.1:5080 <username>@ssh.wvu.edu
```

The second one is a more recent version. Also if you get an error, just try the exact same command again by using the up key. Sometimes you have to run the command like 3 times to get it to work.

5. SSH from the center to the PICARD server using one of Devine's given credentials:

```
ssh -L 5080:localhost:5080 <username>@157.182.194.132
ssh -L 5080:127.0.0.1:5080 <groupname>@157.182.194.132
```

The current PICARD group username is "backend". This may change in later semesters.

6. Once you are on the server you should be in "/home/backend". You can verify this by Printing your Working Directory with the "pwd" command.

6.1 If you aren't in /home/backend, navigate there with "cd /home/backend"

7. Now we need to check if the backend is actually running. You can do this by running "lsof -i :5080" or "ps aux | grep API-backend" . If this command produces any output, then there is currently a local process listening/using port 5080.

```
backend@picard:~$ lsof -i :5080
COMMAND        PID    USER   FD   TYPE    DEVICE SIZE/OFF NODE NAME
API-backe 2930744 backend  308u  IPv4 164265037      0t0  TCP localhost:5080 (LISTEN)
API-backe 2930744 backend  309u  IPv6 164265038      0t0  TCP ip6-localhost:5080 (LISTEN)
```

Example of output if dotnet is already running.

8. If the command in step 7 produced output then you are good to go. You successfully set up the frontend and connected it to the docker-swarm on the backend.

8.1 If step 7 produced no output, run "cd /home/backend/API-backend" to get into the repository and from there run "dotnet run". If you don't get an error message then you should be good to go. This dotnet instance WILL end if the terminal is closed and remove functionality from the website.

Navigating the Website

The website was designed primarily with chrome in mind. Errors, such as, CORS errors may result when attempting to submit an algorithm, experiment, or dataset on a non-chrome browser. You can view these errors through web-console (ctrl+shift+k) on firefox.

1. When you boot up the frontend, it will likely be on localhost/5173. This port was chosen arbitrarily.
2. Click on the login button.
3. You should now be taken to localhost/5080. This is on the backend thanks to your ssh tunnel to 5080.
4. You will be prompted to login with a google account. Make sure that you login with an account ending in @mix.wvu.edu. This process may fail and display a whitepage, 404 error page, or freeze. If this happens, go back a few pages and try again. Only consider the login failed after 4 failed login attempts.
5. You now are at the main dashboard.
6. Logging in using cookies from a previous session may result in odd behavior.

Navigation

1. To run an experiment, first click "NEW EXPERIMENT".
2. From there click "+ Add New Algorithm".
3. Please use our preset algorithms from the algorithms repo.
4. Next, upload any CSV as your data.
5. Then create a new experiment after selecting your algorithm.
6. Now it should run. ☝️☝️☝️☝️☝️

Things that might make it not run.

- Submitting an experiment while one is running.
- Leaving the page where you watch your experiment on your local page.
- Bad algorithm.

If the experiment fails to run, sometimes it will crash the dotnet instance as well and you will have to restart that as well. Failed experiments will be logged on your profile.

Running backend locally

If needed, you can run the backend locally but without most of the swarm functionality. See the readme page on the backend repo for more information. Setting this up is difficult but is very useful for testing. I recommend that each PICARD team has at least one device with this set up.

About

THE PICARD

The PICARD is a platform for algorithm research primarily for students and faculty. It is designed to provide simplified access for researchers working with big data or complex algorithms requiring powerful hardware. It is intended to be an easy solution for students to test these algorithms and get results.

The project was originally inspired by Devine's master thesis on CoDRIFT. As part of this thesis he developed a custom RandomForest algorithm that was run on astronomically large astronomical data. To ensure that he could test this algorithm before his thesis he had to string up 20+ PC's to accumulate enough power to run the algorithm quickly.

This capstone replicates that process with a set of powerful PC's setup in a WVU building that provide the muscle of this entire project. These PC's are connected using Apache Hadoop/Spark in a docker swarm that allows for multiprocessing to speed up algorithms. The libraries used are specifically compatible with Scala.

The website itself provides a UI to interact with the PICARD computers. It uses Google to authenticate users and is built on a REACT framework along with Vite. The website allows users to upload algorithms and datasets which are saved to each user's profile for future use. The parameters for each experiment can be customized each execution and can even be executed in batch. When executed in batch, the algorithms are given a range of parameters and run sequentially with separate outputs.

Now that you have your pitch to bring to conferences and summits, lets get into whats wrong with this capstone. Remember, this is the backend document, when introducing outsiders to the project use the Front-end version.

Issues

WVU's Network

The PICARD servers exist on an isolated network that can only be accessed through the SSH gateway unless you petition them to open a port or something. You will have to communicate with WVU's ITS department to get this done.

Because of this, users have to tunnel to the server in order to run their algorithms. This is not user friendly and requires setup and WVU ssh permission. This process may have been fixed by making a public front-end application by now or not.

Experiment Submission

To create a new experiment there are many steps you have to complete. First you need to create the algorithm and compress it into a .jar file. Then you need to upload your algorithm and indicate the name and type of each parameter as well as the name of the main method.

Next you need to upload your dataset which can only be a CSV. No checks are done about the structure of this CSV so it may break, I haven't tested this.

And now your algorithms and datasets are saved on your profile right? Wrong. There are some situations where your algorithms and datasets may be deleted from your profile. Usually if another user runs an experiment. If this happens the contents are deleted but the references aren't so the website will look fine, but the files won't exist.

Now that you've done that you can run your experiment. Select your algorithm and dataset, set the parameters, and give it a name. Now we reach the annoying hurdle.

Due to the way the back-end is setup, algorithms that work properly on your local device may fail when sent to the backend. This will only happen after the entire swarm is setup which usually takes around 3 minutes. So it takes around 5 minutes each time to determine if your entire experiment setup is correct. And keep in mind that the error could be all the way back in your algorithm submission.

Backend

Messy

Difficult to coordinate local testing environment with remote environment.

Errors may result when pulling git repositories.

Issues

Problems

Why Tunnel

Because of restrictions on WVU's network we are not allowed to leave an open port on the PICARD server without University approval. I have heard that the IT department is actually pretty chill if you visit them a ton and maintain communication but you'll have to find that out for yourself.
? Fixing this would allow us to run PICARD as a website straight off of the backend server
? Website should be secure before we attempt this

Namenodes get orphaned

When an experiment crashes sometimes it says that it could not connect to a namenode. After it occurs enough times it can prevent the entire backend from starting up. This can be fixed with "`docker system prune`" but a standardized fix would be nice.

? Not cleaning up properly. Would only solve results not occurrence.
? Experiments do not crash gracefully. Might get to the root of problem.

Files disappear when alternating users' experiments

References to the files remain but the files are gone.
Just reupload the files and it works fine.
? Cleanup script working too well.

CoDRIFt, Stringop not found

Running CoDRIFt gives a stringop not found error. Indicates that it could not find a required library to execute.
? Docker image not made correctly.
? Was working fine last semester did something get updated?

Could not access nuget cache

Can be fixed by deleting the project and cloning from github again.

Can't exec into docker container, password required

Some student locked them ages ago and graduated XD.
We'll have to reset the docker image from scratch to fix it. 😡
? Where is the file that created that docker component originally

No parameter validation

Devine set a parameter to "Ham Sandwich" and the whole thing broke.
The parameter submission form is dynamically generated which makes things difficult.
? Ditch the parameter form. Why do arguments that are going to be passed as strings anyway need validation?

Solved

Cannot find namenode at master:4080

A node has been orphaned due to some weird error. Use "`docker system prune`" to nuke everything and remove the orphan. Apparently `scripts/cleanup.sh` and `scripts/cleanup-all.sh` was an attempt at working on this issue that did not complete.

Console does not print on frontend

Was fixed with "`docker system prune`". Not sure why this happened hahaha.

Form field cannot be void

Old algorithm submission expects a defaultValue with each parameter. Patched out with an if statement to default defaultvalue to "0" if not supplied.

jszip could not find file

OK so .jar and .zip are the same thing different names. JSZIP unzips jars to check for metadata but will crash if it is a .py or something else. Should be fixed.

Weird tunneling issue.

Use "`5080:127.0.0.1:5080`" instead of "`5080:localhost:5080`" when tunneling.
Some devices can interpret that localhost actually means 127.0.0.1 but sometimes they don't.
? Might be an issue with how `/etc/hosts` is configured on device?

How it works.

Frontend

The frontend is a nodeJS framework that performs basic tasks. It is essentially a GUI for the backend. That doesn't mean it doesn't have its quirks.

Login

The login request is handled entirely by the backend which uses a Google API authentication to log users in. It only checks that the user has an email ending in .mix or that they have a token. I have been told that it is very lenient with the token checking and doesn't examine it thoroughly or expire it. So you can frequently jump past the login page without the backend even running.

Also yes it regularly makes you click the login button twice. IDK why it does that.

Stored Files

The files are probably stored in an SQL database on the backend. They are generated dynamically by the backend. This means that while uploading a file or if the backend is not connected they won't show up on the user's profile. There is a bug where if more than one user run experiments interchangeably the other user's files will be deleted from the backend. They will appear to be present but upon trying to use them, the backend will produce an error. The fix is to reupload them.

Uploading Files

Uploading is done via simple HTML commands. I'm not really sure how they get to the supposed SQL server back there. But I do know that there are checks on the frontend to make sure that the submitted files are a jar, or csv.

Algorithm submission

When you select a file to upload, the file will be parsed via JSZIP. The file structure is searched for a "picard.json" file which will autofill the parameters for the algorithm submission. Otherwise, the upload fails. So if your algorithm is not a jar or a file that JSZIP can't open it won't work.

Also if you fill out the parameters manually it will also fail somewhere along the line because you didn't specify a default value for the form entries. This may or may not have been fixed and pushed by now.

Experiment Execution

Not much to mention here. Fairly standard. You will be able to see some unused buttons and unchangeable fields. Those are remnants of unimplemented features from previous semesters. Also if the "show results" console stops showing output it might be because the backend needs to cleaned with "docker system prune". Or it could be something entirely different :D idk.

Backend

Quite unexplored

Cluster

The cluster is initiated for a docker-swarm creation. To my knowledge there is no difference for the container for the worker, master, and driver nodes. I believe a docker network connects all the nodes but I'm not sure.

Cluster Initialization

After an experiment is submitted by the frontend, several cogs begin to turn. Firstly, the dotnet application processes the request and starts several C# scripts. These scripts process the request and begin building the environment. Along the way some dockerfiles are dynamically changed, I don't really know how or why but they are.

Eventually the Dockerfile is called and the swarm is created. The container is quite old and is finnicky with running commands on it. It comes with Python 2.7 installed. You can try to install a later version of Python but it will not be compatible because of the difference in glibc versions. This is a priority for me to fix so I can freely add modern python libraries.

This Dockerfile initializes each swarm node environment with hadoop and spark. Hadoop is the manager of the nodes, providing an interface for network communication between the independent containers. Spark is the actual executor. It provides distributed processing and optimization when executing functions within its libraries. Particularly .map functions.

Once created, the experiment is submitted via a bash script. This script ensures that the output and input files are accessible while the program runs. It also adds a mount so the output file is exportable once the experiment completes. It then submits the experiment with the system specifics. Things like number of: cores, executors, and memory.

This starts Spark running. If spark crashes it can improperly close, orphaning nodes. These nodes can be cleaned by "docker system prune". And I don't really know what goes on beyond then because it hasn't caused me any problems :).